OmniMesh — System Architecture v2

Oct 24, 2026 · @Suvam

1. Vision

OmniMesh merges the free tiers of many cloud providers into one backend behind one domain, and moves traffic to a healthy server within about 3 seconds when any server fails. It does for servers what OmniRouter does for AI providers: one entry point, many interchangeable backends, automatic failover.

Who it is for: students, indie developers, early startups and open-source maintainers who need a backend that stays up but cannot pay for AWS yet.

What the developer does:

1. Runs `npx omnimesh init` and ticks the accounts they already have (Cloudflare, Koyeb, Render, Vercel, Oracle and so on).
2. Pastes one API token per provider. No credit card? Oracle and R2 are simply skipped.
3. Runs `omnimesh deploy`. The same app is deployed to every provider, and one domain routes to all of them.
4. Runs `omnimesh chaos` to kill a server and watch traffic move to another one live.

Positioning (what the README says): "Your free tiers, merged into one cloud." It is a resilient zero-cost backend for side projects, MVPs and early startups. It is not marketed as unlimited, and not as a replacement for AWS at Flipkart scale.

2. Will it work?

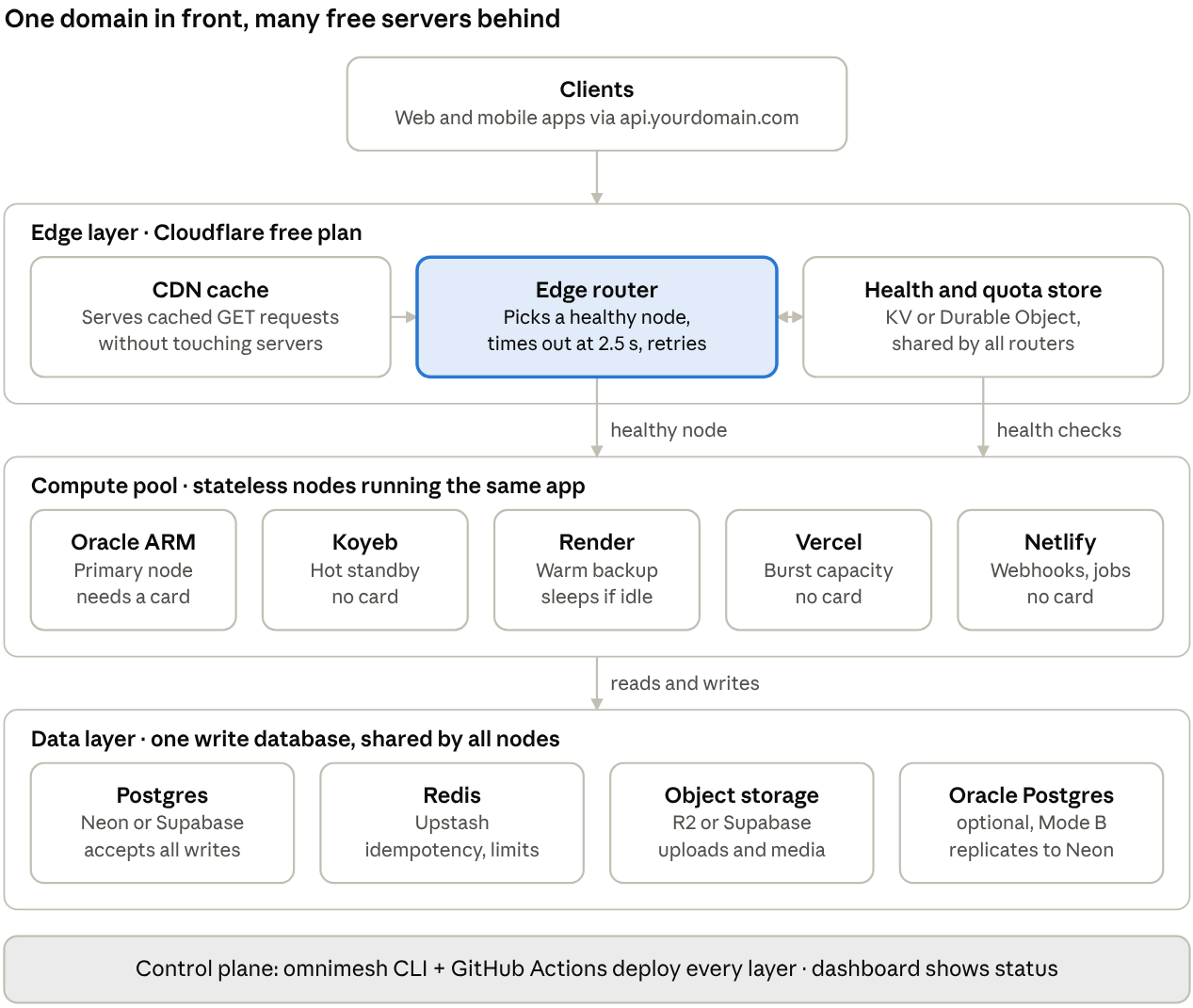
Yes, for stateless compute failover and cached reads. It is harder for the database, and it will not reach Flipkart scale on free tiers. Being clear about this is what makes the repo trusted.

Claim	Realistic?	Why
If one server dies, another answers within ~3 s	Yes	A timeout plus retry at the edge router is simple and proven

Claim	Realistic?	Why
One domain in front of 6 to 8 providers	Yes	Cloudflare DNS, CDN and Workers do this on the free plan
High read traffic at $0	Yes, if cacheable	Public GET responses are served from the Cloudflare cache, not your servers
Database survives any single failure	Partly	Compute is easy to duplicate; one database is still the weak point (section 7)
Unlimited servers	No	Each provider caps requests, RAM and bandwidth; the mesh adds them up, it does not remove limits
Flipkart-scale backend	No	That needs paid infrastructure, many database replicas and a team; OmniMesh is a starting point that can later move to paid tiers

The honest promise: **a backend that does not go down when one free server does, and costs $0 until you outgrow it.**

3. Architecture at a glance



OmniMesh architecture · 4 layers + control plane

Every request enters through one Cloudflare domain; cached reads stop at the CDN, everything else goes to the edge router, which sends it to a healthy compute node that reads and writes one shared data layer.

4. Components

OmniMesh has eight components; only the edge router sits in the request path, everything else supports it.

Component	Runs on	Job
Edge router	Cloudflare Workers (primary), Deno or Bun mirror	Receives every request, picks a healthy node, enforces timeout, retries, returns the answer
Health and quota store	Cloudflare KV or a Durable Object	Shared list of which nodes are down and how much of each quota is used, so every router instance agrees
Health checker	Cron in the router (every 1 min)	Calls `/health` on each node and updates the store before users hit a dead server
Compute nodes	Oracle, Koyeb, Render, Vercel, Netlify	Run the same app image; stateless; verify the gateway secret and JWT
Data layer	Neon or Supabase Postgres, Upstash Redis, optional Postgres on Oracle	The single source of truth for data, plus idempotency keys and rate limits
Object storage	Cloudflare R2 (card) or Supabase Storage (no card)	User uploads, media, static files
Control plane	`omnimesh` CLI + GitHub Actions	Validates tokens, deploys to every provider, writes router config, runs chaos tests
Dashboard	Cloudflare Pages	Live node status, failovers, quota bars, latency

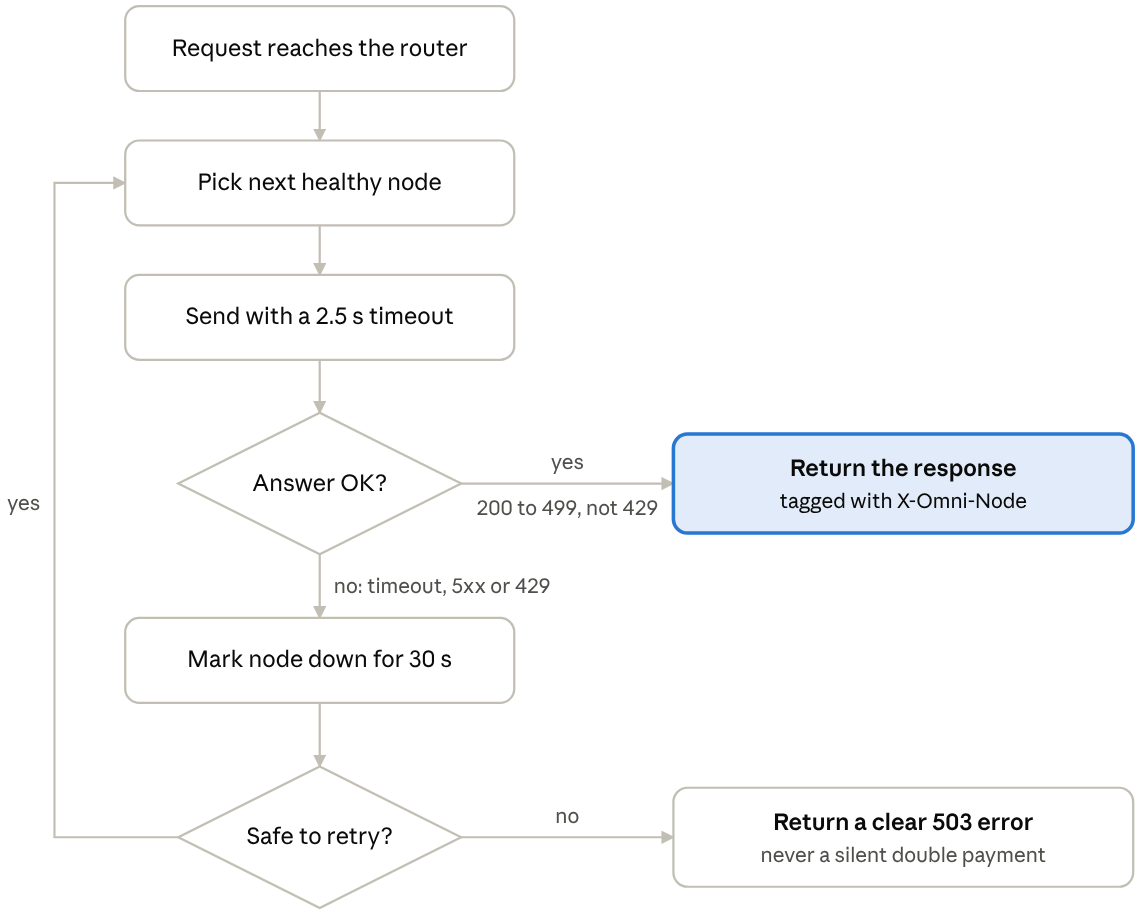
Key design rules

- Compute nodes hold no state. Any node can answer any request.
- The router never guesses: every routing decision reads the shared health and quota store.
- Limits live in config files, not code, because free tiers change.

5. Request lifecycle and failover

When a node fails, the user waits one timeout (about 2.5 s) and then gets a normal answer from the next node; later requests skip the dead node for 30 seconds.

A failed node costs one timeout, not an error page



Safe = GET, or a write with an Idempotency-Key, and under 3 tries and 6 s in total

router failover loop · 2 decisions, 1 retry loop

Changes from the original PDF design

- Health is shared across all router instances (section 4), not kept in one isolate's memory.
- At most 3 attempts and 6 seconds in total, so a full outage fails fast instead of hanging.
- Writes are retried only with an `Idempotency-Key`; without one the router returns the error rather than risk running the action twice.
- Large uploads (over 1 MB by default) are streamed and not retried.

- WebSockets do not fail over mid-connection; the client reconnects and lands on a healthy node.

6. Provider adapters

Every provider is a plugin with the same interface, so the community can add new ones in under an hour. This is the main way the repo grows.

```
interface ProviderAdapter {
  id: string;                    // "render", "koyeb", "oracle"
  requiresCard: boolean;
  tier: "free" | "trial-credit" | "paid";
  roles: ("edge-router" | "compute" | "database" | "cache" | "storage")[];
  limits: ProviderLimits;        // from limits.json, with lastVerified +
sourceUrl
  validateCredentials(cfg): Promise<boolean>;
  deploy(app, cfg): Promise<DeployResult>;
  healthUrl(deployment): string;
  getUsage?(cfg): Promise<UsageSnapshot>;
  destroy(deployment, cfg): Promise<void>;
}
```

Starting catalog. Limits are left out on purpose: they must be checked against each provider's official pricing page when the adapter is written, and stored with a `lastVerified` date.

Provider	Role	Card needed	Note
Cloudflare (DNS, CDN, Workers, KV)	Edge router, cache, health store	No	The backbone; required
Koyeb	Compute (hot standby)	Check current policy	Always-on free service if still offered
Render	Compute (warm or cold backup)	No	Free services sleep when idle
Vercel	Compute (serverless burst)	No	Hobby plan is for non-commercial use

Provider	Role	Card needed	Note
Netlify	Compute (webhooks, functions)	No	Good for async jobs
Deno Deploy	Router mirror, light compute	No	Platform changed recently; verify
Oracle Cloud Always Free	Big compute, optional self-hosted Postgres	Yes	Most power; one account per person
Neon or Supabase	Postgres	No	Shared primary database in Mode A
Upstash	Redis	No	Idempotency keys, rate limits
Cloudflare R2	Object storage	Yes	No egress fees
Supabase Storage	Object storage	No	No-card alternative to R2
Railway, Fly.io	Compute	Trial or paid	No permanent free tier; supported as paid adapters

No-card path: Cloudflare + Koyeb + Render + Vercel + Netlify + Neon + Upstash + Supabase Storage. That is a full working mesh without any card.

6.1 No-card mode (automatic)

If the user has no credit or debit card, OmniMesh skips Oracle and R2 automatically and gives their jobs to free services that need no card, so the whole mesh still runs.

How it works

1. `omnimesh init` asks: "Do you have a card for cloud sign-ups?"
2. **No** → providers that need a card are hidden, and the `no-card` profile is loaded.
3. The router runs on Cloudflare Workers (no card needed) and routes to the no-card nodes below.
4. If the user gets a card later, `omnimesh add-provider oracle` makes Oracle the primary node and moves the others to backup. No rebuild needed.

Job	With a card	Without a card (auto-replacement)
Edge router	Cloudflare Workers	Cloudflare Workers
Router mirror	Deno Deploy	Deno Deploy
Health and quota store	Cloudflare KV	Cloudflare KV
Primary compute	Oracle ARM VM	Koyeb (Render if Koyeb asks for a card)
Backup compute	Koyeb, Render	Render, Vercel
Auth service	Deno, Vercel, Oracle	Deno, Vercel, Netlify
Heavy jobs	Oracle	Queued in Upstash, processed by Koyeb or Render
Webhooks	Netlify, Oracle	Netlify, Vercel
Database	Oracle Postgres (Mode B) or Neon	Neon or Supabase (Mode A)
Cache and idempotency	Upstash or Redis on Oracle	Upstash
File storage	Cloudflare R2	Supabase Storage

Config the wizard writes

```
profile: no-card          # set automatically by omnimesh init
pools:
  /auth:     [deno, vercel, netlify]
  /api:      [koyeb, render, vercel]
  /heavy:    [koyeb, render]   # long jobs go through an Upstash queue
  /webhooks: [netlify, vercel]
database:
  mode: managed
  primary: neon
storage: supabase
```

Trade-off: without Oracle each node has much less RAM and CPU and the database has less storage, so no-card mode suits lighter apps. Heavy CPU work should go through the queue. Card rules can change, so each adapter's `requiresCard` flag is checked when its

limits are verified.

7. Data and database failover

There is always exactly one database that accepts writes; OmniMesh never lets two databases accept writes at once, because they would drift apart and lose orders or payments.

Mode	Who it is for	Primary database	Backup	If the primary fails
A: Managed (default)	Everyone, no card	Neon or Supabase Postgres, pooled connection	Provider's own backups + nightly dump to R2 or Supabase Storage	App goes read-only from cache until the provider recovers
B: Self-hosted	Oracle users needing more storage	Postgres on Oracle VM via Cloudflare Tunnel	Logical replication to Neon (seconds behind, not 5 minutes)	Router switches reads to Neon; writes wait until an `omnimesh db promote` decision

Rules every node follows

- **Idempotency:** every POST or PATCH that must be retried carries an `Idempotency-Key`. The node stores the key and the result in Upstash Redis before running the action, so a retry returns the stored result instead of charging twice.
- **Sessions:** stateless JWTs signed with keys shared by all nodes, so a user stays logged in whichever node answers.
- **Cache:** shared Redis, never in-process memory, so every node sees the same values.
- **Promotion is a human decision by default.** Automatic promotion can be turned on, but the docs explain the risk of losing the last few seconds of writes.

8. Security, ToS and abuse protection

OmniMesh uses one legitimate account per provider and never helps anyone create extra accounts to dodge limits; that rule protects users from bans and the repo from takedowns.

Threat	Protection
Someone finds a raw backend URL and bypasses the router	Router adds `X-Gateway-Secret`; nodes return 403 without it; Oracle has no public ports (Cloudflare Tunnel only)
Weak or default secret	Router and nodes refuse to start without a real secret (fail closed)
A bot burns your free quota	Rate limits per IP and per user at the edge; Cloudflare bot protection; quota-aware routing stops before limits
Double payment on retry	POST and PATCH retried only with an `Idempotency-Key` (section 7)
Leaked tokens	Tokens only in `.env` and GitHub secrets; never logged; secret scanning in CI
Provider ToS violation	One account per provider; keep-alive pings off by default with a warning; PRs that add multi-account or quota-evasion features are rejected (stated in `CONTRIBUTING.md`)
Vulnerable dependencies	Dependency audit and CodeQL in CI

9. Developer experience

A new user should go from zero to a live multi-cloud backend in under 10 minutes with one config file and five commands.

Command	What it does
`npx omnimesh init`	Wizard: pick providers (card badge shown), paste tokens, validate them, write `omnimesh.config.yaml` and `.env`
`omnimesh deploy`	Builds once, deploys to every provider in parallel, deploys the router last, rolls back if health checks fail
`omnimesh status`	Live table of nodes, health, latency, quota used

Command	What it does
`omnimesh chaos`	Takes one node down on purpose and shows the failover time; the demo people share
`omnimesh doctor`	Checks config, tokens, secrets and quotas, and explains how to fix each problem
`omnimesh add-provider <name>`	Adds one provider to an existing mesh
`omnimesh db promote`	Makes the backup database the primary after a failure (Mode B)

Example config

```
domain: api.myapp.com
router:
  timeoutMs: 2500
  maxAttempts: 3
  totalBudgetMs: 6000
  circuitTtlMs: 30000
pools:
  /auth: [deno, vercel, oracle]
  /api:  [oracle, koyeb, render, vercel]
  /webhooks: [netlify, oracle]
database:
  mode: managed        # managed | self-hosted
  primary: neon
cache: upstash
storage: r2            # r2 | supabase
keepAlive: false       # off by default; check each provider's terms
```

CI/CD: one GitHub Actions workflow runs tests on every push to `main`, then calls `omnimesh deploy`.

10. Repo, stack and roadmap

Build it as a TypeScript monorepo and ship a working failover demo first; everything else follows.

Stack: TypeScript, pnpm + Turborepo, Hono (one router codebase for Workers, Deno, Bun and Node), Zod for config, Vitest for tests, k6 for load tests, Starlight or VitePress for docs. License: MIT.

```
omnimesh/
├── packages/
│   ├── router/        # edge router (Hono)
│   ├── node/          # @omnimesh/node middleware for compute nodes
│   ├── core/          # config schema, shared types
│   ├── cli/           # npx omnimesh
│   ├── dashboard/     # status dashboard
│   └── providers/     # one folder per provider + _template
├── examples/          # rest-api, auth, webhooks, uploads
├── benchmarks/        # load + chaos tests and published results
├── docs/
├── .github/workflows/ # ci.yml, deploy-mesh.yml, release.yml
├── README.md  CONTRIBUTING.md  SECURITY.md  LICENSE
```

Build order

1. **Router core:** failover, timeouts, circuit breaker, retry rules, tests against 3 local mock servers. Gate: kill one mock, every request still succeeds.
2. **Node middleware:** gateway secret, JWT, idempotency, `/health`.
3. **No-card adapters:** Cloudflare, Koyeb, Render, Vercel, Netlify, Neon, Upstash.
4. **CLI:** `init`, `deploy`, `status`, `doctor`, `chaos`. Gate: a fresh user deploys in under 10 minutes.
5. **Shared health store and quota-aware routing.**
6. **Card adapters:** Oracle (with Mode B database), R2.
7. **Dashboard and benchmarks:** publish measured failover time and p95 latency.
8. **Launch:** README, docs, demo GIF, v1.0 release.

Launch checklist (how it goes viral)

- [ ] README opens with a 10-second GIF: a server is killed, traffic moves, the request still succeeds
- [ ] 3-command quickstart and a "Deploy with zero credit card" section
- [ ] Provider table with card badge and `lastVerified` dates
- [ ] Honest "Limits" section (section 2 of this doc)
- [ ] Real benchmark numbers, with the script to reproduce them
- [ ] 10+ "good first issue" tickets, each a new provider adapter
- [ ] Launch posts: Show HN, r/selfhosted, r/webdev, dev.to, X, LinkedIn, Indian dev communities